

Practical 1: Designing and Implementing a Multi-Zone DMZ

Objective: Students must configure a firewall to protect a three-tiered application (Web, App, and Database) using a "Three-Homed" architecture.

1. The Scenario

The enterprise "EduSecure" has a web server that must be accessible to the public. However, the web server needs to pull data from a private SQL database. The database must never be directly accessible from the internet.

2. Tools Required

- GNS3 or Cisco Packet Tracer: To simulate the network topology.
- pfSense or Cisco ASA Image: A virtual firewall appliance.
- Any Linux Distro (Ubuntu/Kali): To act as the "External Attacker" and "Internal User."

3. Task Requirements (10 Marks)

1. Topology Setup: Create three distinct security zones:
 - WAN (Outside): 192.168.1.0/24
 - DMZ (Public Servers): 172.16.1.0/24
 - LAN (Private Inside): 10.0.1.0/24
2. Rule Configuration: Implement the following logic:
 - Rule A: Allow HTTP/HTTPS (80/443) from WAN to DMZ Web Server.
 - Rule B: Allow SQL traffic (1433) from DMZ Web Server to LAN Database.
 - Rule C: Deny ALL traffic initiated from WAN to LAN.
 - Rule D: Deny ALL traffic initiated from DMZ to WAN (to prevent data exfiltration if the web server is hacked).

Verification: Use nmap from the WAN node to prove that Port 80 is open on the DMZ but Port 1433 is "Filtered/Closed"

REPORT

PRACTICAL 1

INTRODUCTION:

This practical focuses on designing and implementing a secure multi-zone network architecture using Cisco Packet Tracer. A three-tier architecture consisting of WAN, DMZ, and LAN zones was created, and Access Control Lists (ACLs) were configured on a router to enforce security policies and control traffic flow between the zones.

OBJECTIVE:

The objective of this practical is to design a secure network with three zones (WAN, DMZ, and LAN), configure a router with ACLs to control communication between the zones, and ensure that sensitive resources such as the database server are protected from direct external access

NETWORK DESIGN:

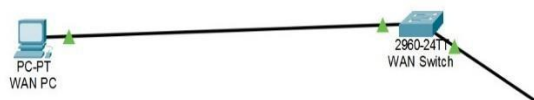
• Zone Network Address Description

WAN 192.168.1.0/24 External/Public Network

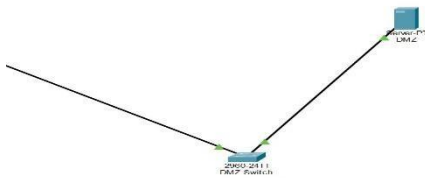
DMZ 172.16.1.0/24 Web Server Zone

LAN 10.0.1.0/24 Internal Private Network

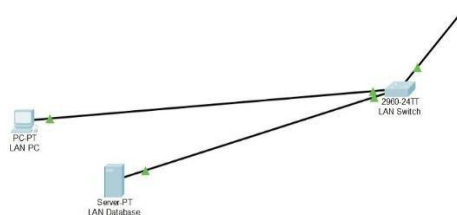
All shown in the *images below*



WAN ZONE (a switch and a PC)



DMZ ZONE (a server connected to switch)



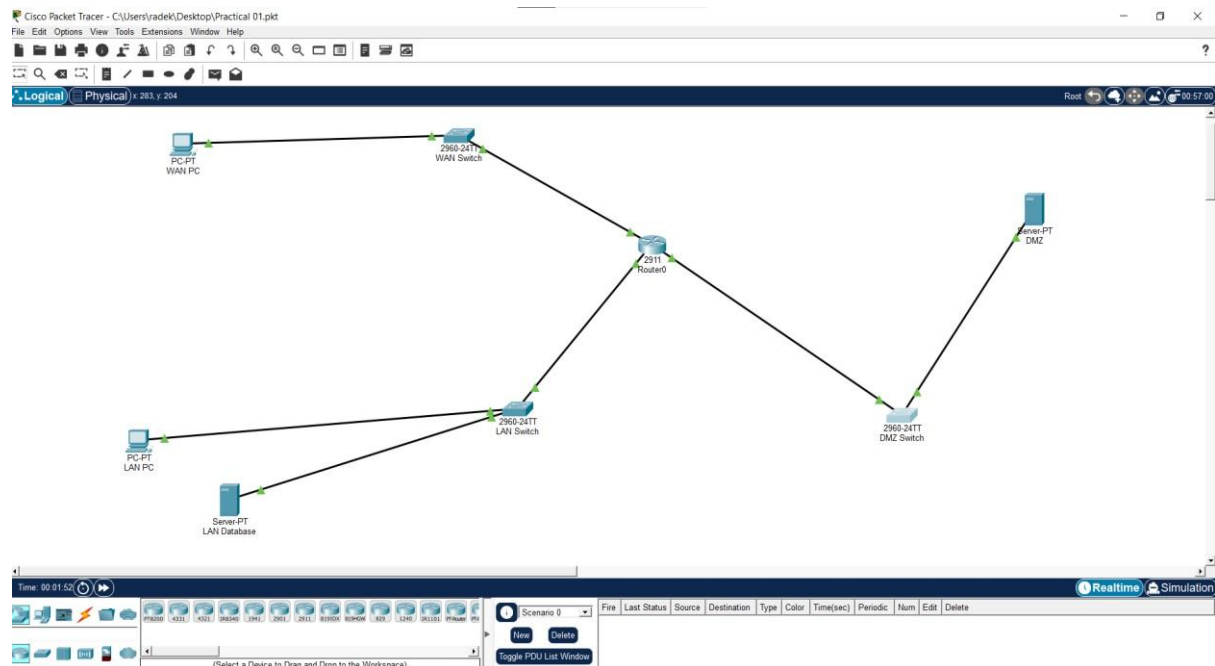
LAN ZONE (a server and a PC connected to a switch) **Network Topology Description**

The network consists of a router connecting three different zones:

The **WAN** represents external users (attacker/public)

The **DMZ** contains a web server accessible to the public

The **LAN** contains a private database server and a Personal computer



The image above shows the WAN zone at top left side of the screen, LAN zone at the down left side of the screen and the DMZ zone at the right side of the screen, all connected to the router placed at the middle.

4. IP ADDRESSING

- **Device Interface IP Address**

At this stage, The Router and the End devices were all allocated IP addresses to proceed to the next stage of connection;

Router (WAN) Gig0/0 192.168.1.1

Router (DMZ) Gig0/1 172.16.1.1

Router (LAN) Gig0/2 10.0.1.1

Web Server (DMZ) - 172.16.1.20/24

Database Server (LAN)- 10.0.1.30/24

Attacker PC (WAN) - 192.168.1.10/24

5. CONFIGURATION

This is the point where the router is configured by allocating IP addresses to each interfaces to enforce the ACLs to be entered.

- **Router Interface Configuration**

```
interface g0/0 ip address 192.168.1.1
```

```
255.255.255.0 no shutdown interface
```

```
g0/1 ip address 172.16.1.1
```

```
255.255.255.0 no shutdown
```

```
interface g0/2 ip address 10.0.1.1
```

```
255.255.255.0 no shutdown
```

- **ACL Configuration**

This is the stage where the ACLs are configured at the router interface.

Note: ACL > Access Control Lists.

To do this, we clicked on the router, navigated to CLI, and the entered these codes;

```
Router> enable
```

```
Router# configure terminal
```

! Rule A: Allow HTTP/HTTPS from WAN → DMZ Web Server

```
access-list 100 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.10 eq 80 access-
```

```
list 100 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.10 eq 443 ! Rule B:
```

Allow SQL traffic (1433) from DMZ Web Server → LAN Database access-

```
list 100 permit tcp 172.16.1.10 0.0.0.0 host 10.0.1.10 eq 1433 ! Rule C: Deny
```

ALL traffic initiated from WAN → LAN access-list 100 deny ip 192.168.1.0

```
0.0.0.255 10.0.1.0 0.0.0.255 ! Rule D: Deny ALL traffic initiated from DMZ
```

```
→ WAN access-list 100 deny ip 172.16.1.0 0.0.0.255
```

! Default Rule:

```
access-list 100 permit ip any any
```

- **Apply ACL to the router interfaces**

This is done also at the router's CLI to make the router recognise the interfaces involved in the ACLs.

```
Router(config)# interface gigabitEthernet0/0
```

```
Router(config-if)# ip access-group 100 in
```

```
Router(config-if)# exit
```

```
Router(config)# interface gigabitEthernet0/1
```

```
Router(config-if)# ip access-group 100 in
```

```
Router(config-if)# exit
```

```
Router(config)# interface gigabitEthernet0/2
```

```
Router(config-if)# ip access-group 100 in
```

```
Router(config-if)# exit
```

6. SECURITY POLICY IMPLEMENTATION

The following rules were implemented:

Allow HTTP/HTTPS from WAN to Web Server (DMZ): HTTP(S) traffic from WAN to Web Server was allowed through port 80 and 443

Allow SQL traffic from Web Server to Database: Traffic was allowed through port 1433

Deny all traffic from WAN to LAN: All traffic were denied from WAN to LAN

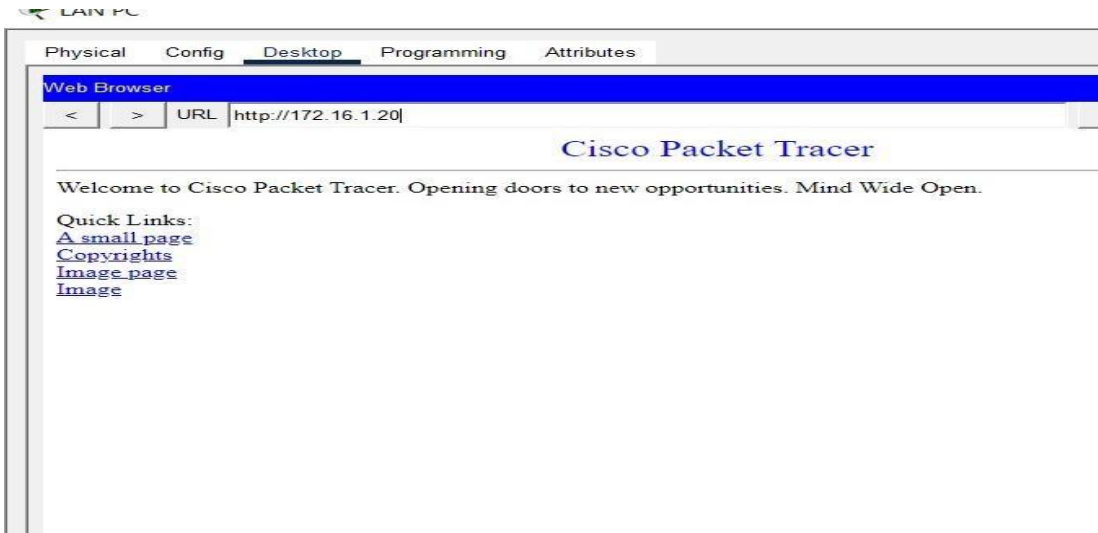
Deny traffic from DMZ to WAN: All traffic were denied from DMZ to WAN

Allow all other safe traffic: All traffics not included in the ACL were allowed.

7. TESTING AND VERIFICATION

- Test 1: WAN to Web Server:

HTTP access from WAN to DMZ web server was successful.



- **Test 2: DMZ to Database**

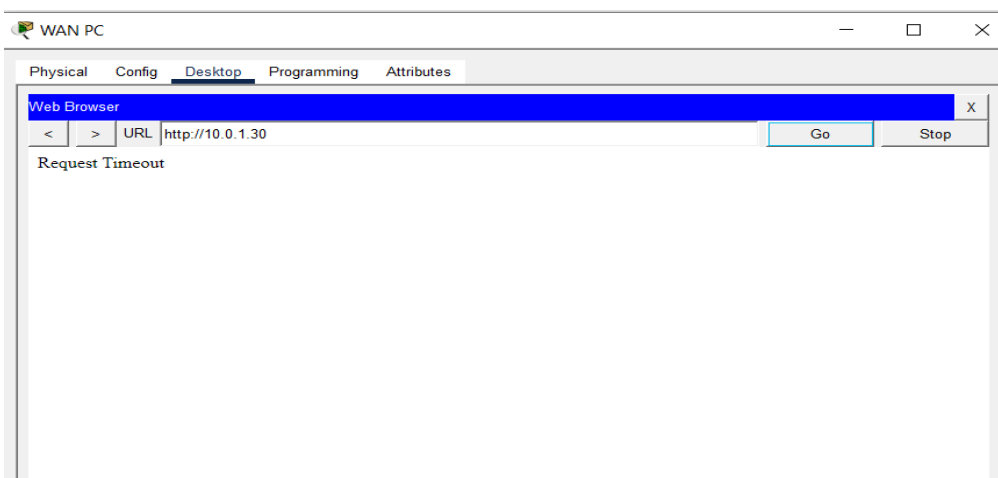
The web server successfully communicated with the database server via port 1433.

```
tended IP access list 101
10 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.20 eq www
20 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.20 eq 443
30 permit tcp host 172.16.1.20 host 10.0.1.30 eq 1433
40 deny ip 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
50 deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255
60 permit ip any any
```

- Since Cisco packet tracer does not support SQL Traffic, we could only provide the screenshot of the ACLs to confirm that this Rule was added.

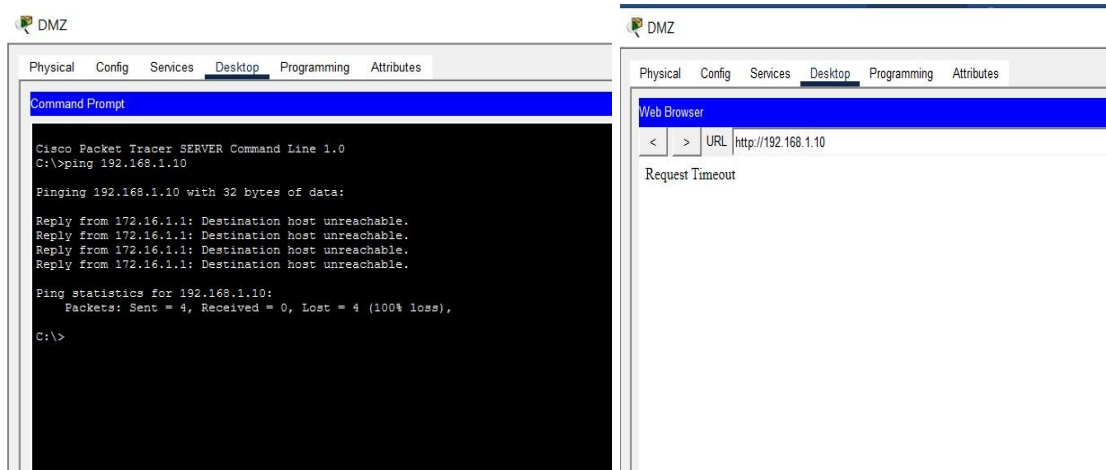
- **Test 3: WAN to Database Server**

Direct access from WAN to LAN database server was blocked, confirming proper security implementation as shown below;



- **Test 4: DMZ to WAN**

All traffic from DMZ to WAN were denied as shown below



8. RESULTS

The implemented ACL successfully enforced network segmentation and security policies. Public users were able to access the web server, while direct access to the database server was restricted. The system effectively prevented unauthorized access and ensured controlled communication between network zones.

9. CONCLUSION

This practical demonstrated how network segmentation and ACLs can be used to secure enterprise environments. By implementing a multi-zone architecture, sensitive systems were protected while still allowing necessary communication. This approach is widely used in realworld network security.

10. Verification

Since Verifying the port 80 and 443 using nmap is impossible on Cisco packet tracer, we'll refer back to the confirmation of **Rule A, (which allows http traffic)** and the Access Control Lists showed in the terminal

```
tended IP access list 101
10 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.20 eq www
20 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.20 eq 443
30 permit tcp host 172.16.1.20 host 10.0.1.30 eq 1433
40 deny ip 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
50 deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255
60 permit ip any any
```

Practical 2: Firewall Rule-Base Optimization & Log Analysis

1. INTRODUCTION

This practical focuses on analyzing and optimizing firewall rules to eliminate conflicts such as shadow rules, improve performance, and implement logging for monitoring blocked traffic. The firewall was configured using “**iptables**” on a Linux system.

2. OBJECTIVE

The objective of this practical is to identify errors in a faulty firewall rule set, correct the rule order using best practices, optimize performance using stateful inspection, and implement logging to monitor denied traffic.

3. PROBLEM ANALYSIS (VERY IMPORTANT)

The given firewall script contained a major issue known as a shadow rule. The rule "**PERMIT ANY ANY ANY**" allows all traffic and prevents subsequent rules from being executed due to the top-down processing nature of firewalls. As a result, important security rules such as blocking the guest network and allowing specific hosts were ignored.

4. CORRECTED FIREWALL RULES

```
iptables -A INPUT -m conntrack --ctstate  
ESTABLISHED, RELATED -j ACCEPT  
iptables -A INPUT -s 192.168.10.0/24 -j DROP  
iptables -A INPUT -s 10.0.0.5 -j ACCEPT  
iptables -A INPUT -d 10.0.0.100 -p tcp -  
-dport 80 -j ACCEPT iptables -A  
INPUT -j LOG --log-prefix  
"DROP: " iptables -A  
INPUT -j DROP
```

5. OPTIMIZATION TECHNIQUE

A stateful rule was added using the conntrack module to allow established and related connections. This reduces processing overhead by preventing already accepted connections from traversing the entire rule set.

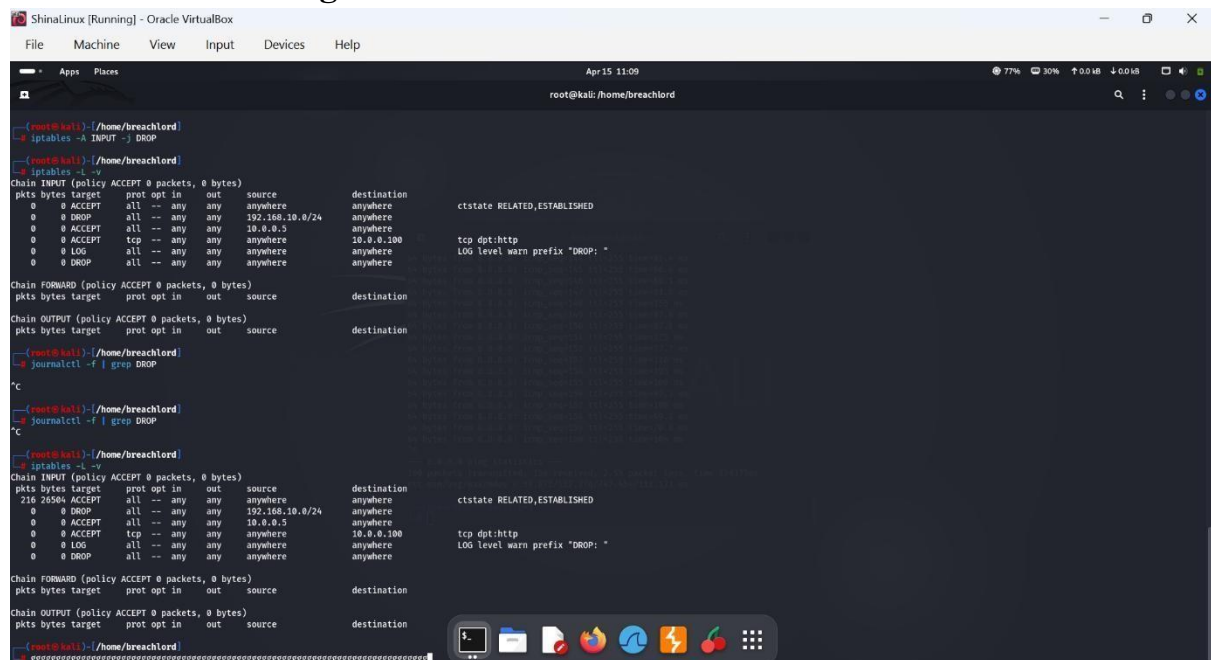
6. LOGGING IMPLEMENTATION

Logging was implemented using the LOG target in iptables with a custom prefix "DROP:". Since Kali Linux uses “**systemd**” instead of traditional syslog, logs were monitored using the “**journald**” command.

7. TESTING AND VERIFICATION

To verify the firewall configuration, ICMP traffic (ping) was sent to an external IP address (8.8.8.8). The firewall successfully blocked the traffic and generated log entries, confirming that the rules were correctly implemented.

Firewall Rules Configuration



```
(root@kali) ~/home/breachlord
→ iptables -A INPUT -j DROP

(root@kali) ~/home/breachlord
→ iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination ctstate
0 0 ACCEPT all -- any any anywhere anywhere ctstate RELATED,ESTABLISHED
0 0 DROP all -- any any 192.168.10.0/24 anywhere
0 0 ACCEPT all -- any any 10.0.0.5 anywhere
0 0 ACCEPT tcp -- any any anywhere 10.0.0.100 tcp dpt:http
0 0 LOG all -- any any anywhere anywhere LOG level warn prefix "DROP: "
0 0 DROP all -- any any anywhere anywhere

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

(root@kali) ~/home/breachlord
→ journalctl -f | grep DROP
^C
(root@kali) ~/home/breachlord
→ journalctl -f | grep DROP
^C
(root@kali) ~/home/breachlord
→ iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination ctstate
216 26504 ACCEPT all -- any any anywhere anywhere ctstate RELATED,ESTABLISHED
0 0 DROP all -- any any 192.168.10.0/24 anywhere
0 0 ACCEPT all -- any any 10.0.0.5 anywhere
0 0 ACCEPT tcp -- any any anywhere 10.0.0.100 tcp dpt:http
0 0 LOG all -- any any anywhere anywhere LOG level warn prefix "DROP: "
0 0 DROP all -- any any anywhere anywhere

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination

(root@kali) ~/home/breachlord
→
```

[illegible]

```
ShinaX [Running] - Oracle VirtualBox
```

```
File Machine View Input Devices Help
```

```
Apr 15 11:36 72% 33% 0.0 KB 4.0 MB
```

```
breachlord@kali: ~$ ping -c 3 8.8.8.8
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
```

```
64 bytes from 8.8.8.8: icmp_seq=137 ttl=255 time=72.7 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=138 ttl=255 time=161 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=139 ttl=255 time=84.4 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=140 ttl=255 time=78.7 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=141 ttl=255 time=83.9 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=142 ttl=255 time=70.7 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=143 ttl=255 time=88.6 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=144 ttl=255 time=91.4 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=145 ttl=255 time=66.6 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=146 ttl=255 time=88.1 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=147 ttl=255 time=81.8 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=148 ttl=255 time=159 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=149 ttl=255 time=87.8 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=150 ttl=255 time=87.1 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=151 ttl=255 time=123 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=152 ttl=255 time=77.7 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=153 ttl=255 time=126 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=154 ttl=255 time=193 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=155 ttl=255 time=180 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=156 ttl=255 time=87.3 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=157 ttl=255 time=188 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=158 ttl=255 time=69.1 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=159 ttl=255 time=70.8 ms
```

```
64 bytes from 8.8.8.8: icmp_seq=160 ttl=255 time=184 ms
```

```
^C
```

```
--- 8.8.8.8 ping statistics ---
```

```
160 packets transmitted, 156 received, 2.5% packet loss, time 15937ms
```

```
rtt min/avg/max/mdev = 33.672/137.376/747.454/110.171 ms
```

```
breachlord@kali: ~$ ping -c 3 10.0.0.200
```

```
PING 10.0.0.200 (10.0.0.200) 56(84) bytes of data:
```

```
^C
```

```
--- 10.0.0.200 ping statistics ---
```

```
3 packets transmitted, 0 received, 100% packet loss, time 2039ms
```

The firewall successfully blocked unauthorized traffic and logged all denied packets. The corrected rule order eliminated shadowing issues, and the system performed efficiently due to the optimization rule.

9. CONCLUSION

This practical demonstrated the importance of proper firewall rule ordering, optimization using stateful inspection, and the use of logging for monitoring network security events.

10. VERIFICATION

Wireshark was used to capture ICMP traffic generated during testing. The captured packets showed the source IP address (10.0.2.15) and destination IP address (8.8.8.8). This confirmed that network traffic was being generated and sent, even though the firewall blocked the communication.

